iLearn Practice Exam

11BR · Chapter / Topic 4

Cambridge AS Level Computer Science 9618

Paper 1 Theory Fundamentals

Topic 4: Processor Fundamentals

Name: _____

Class: _____

Date: _____

Time allowed: 1 hour 30 minutes

TOTAL MARKS: 75

Marks obtained: ..... / 75

INSTRUCTIONS

· Answer all questions.
· Write your answers in the spaces provided on this paper, using a black or dark blue pen.
· Do not use an erasable pen or correction fluid.
· No calculators are allowed.
· Do not write anything in the margins.

INFORMATION

· The total mark for this paper is 75.
· The number of marks for each question or part question is shown in brackets [ ].
· This paper contains 8 questions, all drawn from Cambridge past papers on this chapter.
· After you finish, mark your work against the official Cambridge mark scheme (see the source list on the last page).

Questions reproduced from Cambridge International past papers © Cambridge University Press & Assessment, for classroom practice.

1 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	**Operand**	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
SUB	#n/Bn/&n	Subtract the number n from the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
INC	<register>	Add 1 to the contents of the register (ACC or IX)
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are shown:

Address	Data
48	51
49	6
50	48
51	50
52	49
53	50
54	6

The current contents of the ACC and IX are shown:

ACC	2
IX	50

Complete the table by writing the content of the ACC after each program has run.

Program number	Code	ACC content

1	`LDM #50`
	`INC ACC`
	`SUB #1`	

2	`LDI 51` `ADD 52`	

3	`LDR #2`
	`LDX 50`
	`DEC ACC`	

4	`LDD 52`
	`SUB 54`
	`INC ACC`	

[4]

(b) The processor includes these bit manipulation instructions:

Instruction		Explanation
Opcode	**Operand**	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
LSL	#n	Bits in ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

The current contents of memory are shown:

Address	Data
100	00001101
101	10111110
102	11110011
103	00110111
104	00000000

The current content of the ACC is shown:

1	1	1	1	1	1	1	1

Complete the table by writing the content of the ACC after each instruction has run.

The binary number 11111111 is reloaded into the ACC before each instruction is run.

Instruction number	Instruction	ACC content
1	`LSL #2`	
2	`XOR 100`	
3	`AND 103`	

[3]

2 **(a)** The table has six statements about the Von Neumann model for a computer system. Three of the statements are incorrect.

Statement number	Statement
1	The Program Counter (PC) stores the next instruction to be fetched from memory.
2	The Arithmetic and Logic Unit (ALU) performs mathematical and logical operations.
3	The Control Unit (CU) sends signals to other components on the data bus.
4	The Memory Data Register (MDR) transfers data to the memory address stored in the Memory Address Register (MAR).
5	The MAR stores an address from memory.
6	The Accumulator (ACC) stores the result of calculations.

Complete the table by writing the **three** incorrect statement numbers and the corrected statements.

Incorrect statement number	Corrected statement
.....	....
.....	....
.....	....

[3]

(b) Registers that are used in the Fetch-Execute (F-E) cycle include the PC, MAR, MDR and the ACC.

Identify **one** other register and describe its role in the Fetch-Execute (F-E) cycle.

Register .....

Role .....

.....

[2]

(c) Explain how an interrupt from an input device will be detected and handled in the F-E cycle.

.....

.....

.....

.....

.....

.....

.....

..... [4]

3 **(a)** The following table shows part of the instruction set for a processor. The processor has one register: the Accumulator (ACC).

Instruction		Explanation
Opcode	**Operand**	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDM	#n	Immediate addressing. Load the number n to ACC
STO	<address>	Store the contents of ACC at the given address
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
INC	<register>	Add 1 to the contents of the register (ACC)
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
JMP	<address>	Jump to the given address
END		Return control to the operating system

ACC denotes Accumulator
<address> can be an absolute or a symbolic address
denotes a denary number, e.g. #123
B denotes a binary number, e.g. B01001010
& denotes a hexadecimal number, e.g. &4A

The current contents of memory are:

Address	Instruction
10	12
11	11
12	10
13	22
14	22
…	
100	LDD 10
101	ADD 12
102	STO 11
103	CMP 14
104	JPE 107
105	INC ACC
106	JMP 101
107	STO 10
108	END

(i) Trace the program currently in memory using the following trace table.

Instruction address	ACC	Memory address				
		10	11	12	13	14
		12	11	10	22	22

[3]

(ii) State the effect of changing instruction `LDD 10` in address 100 to `LDM #10`

.....

.....

..... [1]

(iii) Identify **and** describe **one** mode of addressing **not** given in the table of instructions in part **(a)**.

Mode of addressing .....

Description .....

.....

.....

[2]

(b) The table shows part of the instruction set for a processor. The processor has one register: the Accumulator (ACC).

Instruction		Explanation
Opcode	**Operand**	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(i) The ACC currently contains the following binary value.

1	1	1	1	0	0	0	0

Write the result after the instruction `OR B00001111` is run.

[1]

(ii) The ACC currently contains the following binary value.

0	0	0	1	1	1	0	1

Write the result after the instruction `XOR #30` is run.

[1]

4 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and an Index Register (IX).

Instruction		Explanation
Opcode	**Operand**	
LDM	#n	Immediate addressing. Load the number n to ACC
STO	<address>	Store the contents of ACC at the given address
ADD	#n/Bn/&n	Add the number n to the ACC
ADD	<address>	Add the contents of the given address to the ACC
SUB	<address>	Subtract the contents of the given address from the ACC
SUB	#n/Bn/&n	Subtract the number n from the ACC
DEC	<register>	Subtract 1 from the contents of the register (ACC or IX)
JMP	<address>	Jump to the given address
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMP	#n	Compare the contents of ACC with number n
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
END		Return control to the operating system
ACC denotes Accumulator <address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

(a) The current contents of memory are:

Address	Instruction
50	47
51	48
52	49
53	50
...	
500	LDM #50
501	DEC ACC
502	CMP 51
503	JPE 505
504	JMP 501
505	STO 50
506	SUB #10
507	STO 51
508	END

(i) Trace the program currently in memory using the following trace table.

Instruction address	ACC	Memory address			
		50	**51**	**52**	**53**
		47	48	49	50

[3]

(ii) Complete the table by identifying **and** describing **two** modes of addressing that are **not** used in the program in part **(a)**.

Mode of addressing	Description
.....	.....
.....	.....
	.....
	.....
.....	.....
.....	.....
	.....
	.....

[4]

(b) The table shows part of the instruction set for a processor. The processor has one register: the Accumulator (ACC).

Instruction		Explanation
Opcode	**Operand**	
AND	#n/Bn/&n	Bitwise AND operation of the contents of ACC with the operand
AND	<address>	Bitwise AND operation of the contents of ACC with the contents of <address>
XOR	#n/Bn/&n	Bitwise XOR operation of the contents of ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of ACC with the contents of <address>
OR	#n/Bn/&n	Bitwise OR operation of the contents of ACC with the operand
OR	<address>	Bitwise OR operation of the contents of ACC with the contents of <address>
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123 B denotes a binary number, e.g. B01001010 & denotes a hexadecimal number, e.g. &4A		

Write the bit manipulation instructions that can be used to set only the most significant bit to 1 in an 8-bit register.

The instructions need to work on a register that contains any 8-bit binary number.

.....

..... [2]

5 The following table shows part of the instruction set for a processor. The processor has two registers, the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	**Operand**	
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
STO	<address>	Store the contents of ACC at the given address
ADD	#n	Add the denary number n to the ACC
JMP	<address>	Jump to the given address
INC	<register>	Add 1 to the contents of the register (ACC or IX)
CMP	<address>	Compare the contents of ACC with the contents of <address>
CMI	<address>	Indirect addressing. The address to be used is at the given address. Compare the contents of ACC with the contents of this second address
JPE	<address>	Following a compare instruction, jump to <address> if the compare was True
IN		Key in a character and store its ASCII value in ACC
OUT		Output to the screen the character whose ASCII value is stored in ACC
END		Return control to the operating system
<address> can be an absolute or a symbolic address # denotes a denary number, e.g. #123		

(a) The instructions in the processor's instruction set can be grouped according to their function.

Identify the instruction group for each of the following opcodes.

IN .....

ADD .....

JPE .....

CMI .....

[4]

(b) The current contents of main memory and selected values from the ASCII character set are given on page 15.

Trace the program currently in memory using the trace table when the input is '1'.

Address	Instruction
10	LDR #0
11	IN
12	STO 101
13	LDX 110
14	CMP 100
15	JPE 21
16	LDD 101
17	ADD #16
18	INC IX
19	STO 100
20	JMP 13
21	OUT
22	END
...	
100	0
101	0
...	
110	51
111	65

ASCII value	Character
49	1
50	2
51	3
52	4
...	
65	A
66	B
67	C
68	D

Instruction address	ACC	IX	Memory address				Output
			100	101	110	111	
			0	0	51	65	

[4]

6 A computer system is designed using the basic Von Neumann model.

Registers and buses are components in the Von Neumann model.

(a) (i) Identify **three other** components in the Von Neumann model of a computer system.

Do not include registers or buses in your answers.

1 .....

2 .....

3 .....

[3]

(ii) Identify **two** differences between special purpose registers and general purpose registers.

1 .....

.....

2 .....

.....

[2]

(b) The following incomplete table contains steps of the Fetch-Execute (F-E) cycle and their descriptions.

Complete the table by writing the missing steps using register transfer notation **and** the missing descriptions.

Step	Description
.....	The address in PC is incremented.
MDR ← [[MAR]]	....
MAR ← [PC]	....
.....	The contents of MDR are copied into CIR.

[4]

(c) Interrupts can be caused by software programs or hardware devices.

State **one** cause of a software interrupt.

.....

..... [1]

(d) The following statements describe the stages that the CPU performs when an interrupt is detected.

There are **three** missing statements.

Write the letter of the missing statements from the table in the correct place to complete the description.

1 At the end of each Fetch-Execute (F-E) cycle, the processor checks if an interrupt flag is set.

2 .....

3 If the interrupt priority is high enough, the processor saves the current contents of the registers.

4 .....

5 When servicing of the interrupt is complete, the processor restores the registers.

6 .....

Letter	Stage
A	The address of the Interrupt Service (ISR) handling routine is loaded into the Program Counter (PC).
B	Lower priority interrupts are re-enabled.
C	The device causing the interrupt transfers data to the CPU.
D	The processor identifies the source of the interrupt and checks the priority of the interrupt.
E	The ISR is incremented.

[3]

7 **(a)** **(i)** State what is meant by relative addressing.

.....

.....

..... [1]

(ii) Registers such as the Accumulator (ACC) and the Index Register (IX) are used in the CPU.

Identify **two** special purpose registers used in the CPU. Do **not** include the ACC or IX in your answers.

1 .....

2 .....
[2]

(b) The following table shows part of the instruction set for a processor. The processor has two registers: the ACC and an IX.

Instruction		Explanation
Opcode	**Operand**	
LDM	#n	Immediate addressing. Load the number n to ACC
LDD	<address>	Direct addressing. Load the contents of the location at the given address to ACC
LDI	<address>	Indirect addressing. The address to be used is at the given address. Load the contents of this second address to ACC
LDX	<address>	Indexed addressing. Form the address from <address> + the contents of the index register. Copy the contents of this calculated address to ACC
LDR	#n	Immediate addressing. Load the number n to IX
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #127		

The current contents of the main memory and the index register are shown.

Address	Instruction
98	8
99	16
100	3
101	98
102	32
IX	2

Write the contents of the ACC after each instruction is executed.

Instruction	Value in ACC
LDM #98	
LDI 101	
LDX 100	

[3]

(c) A student buys a new computer. The table shows the specifications of the old computer and the new computer.

Old computer	New computer
1.8 GHz dual core processor	2.3 GHz dual core processor
16 MB cache	32 MB cache

Explain why increasing the clock speed **and** increasing the cache memory will improve the performance of the computer.

Clock speed .....

.....

.....

.....

Cache memory .....

.....

.....

.....

[4]

8 The following table shows part of the instruction set for a processor. The processor has two registers: the Accumulator (ACC) and the Index Register (IX).

Instruction		Explanation
Opcode	**Operand**	
LDM	#n	Immediate addressing. Load the number n to the ACC
AND	#n / Bn / &n	Bitwise AND operation of the contents of the ACC with the operand
AND	<address>	Bitwise AND operation of the contents of the ACC with the contents of <address>
XOR	#n / Bn / &n	Bitwise XOR operation of the contents of the ACC with the operand
XOR	<address>	Bitwise XOR operation of the contents of the ACC with the contents of <address>
OR	#n / Bn / &n	Bitwise OR operation of the contents of the ACC with the operand
OR	<address>	Bitwise OR operation of the contents of the ACC with the contents of <address>
CMP	#n	Compare the contents of the ACC with number n
CMP	<address>	Compare the contents of the ACC with the contents of <address>
LSL	#n	Bits in the ACC are shifted logically n places to the left. Zeros are introduced on the right-hand end
LSR	#n	Bits in the ACC are shifted logically n places to the right. Zeros are introduced on the left-hand end
<address> can be an absolute or symbolic address # denotes a denary number, e.g. #127 B denotes a binary number, e.g. B10010001 & denotes a hexadecimal number, e.g. &4A		

The current contents of main memory are shown:

Address	Data
100	0000 0011
101	1010 1110
102	1100 1100
103	1111 1111
104	1100 1100

(a) Complete the table by writing the contents of the ACC after the execution of each instruction.

Current contents of the ACC	Instruction	Contents of the ACC after the execution of the instruction
0000 1111	AND 101	
0000 0000	LDM #100	
0000 0001	XOR &F1	
0001 0001	CMP 101	

[4]

(b) The Von Neumann model for a computer system uses registers.

Describe the role of the Memory Address Register (MAR) and Memory Data Register (MDR) in the fetch-execute (F-E) cycle.

.....

.....

.....

.....

.....

.....

.....

..... [4]

(c) Assembly language instructions are grouped.

Complete each statement by writing the name of the appropriate instruction group.

Loading data into the accumulator is an example of an instruction in the

..... group. Incrementing the index register is an example of an instruction in the ..... group. Branching to another address is an example of an instruction in the ..... group. [3]

END OF QUESTION PAPER

Source of each question (for marking)

Find the matching official Cambridge mark scheme for the series/paper shown, to mark your answers.

Q	Past paper	Original question	Marks
1	May/June 2024 Paper 13	Q3 (a),(b)	7
2	May/June 2025 Paper 12	Q1 (a),(b),(c)	9
3	May/June 2025 Paper 12	Q7 (a),(b)	8
4	May/June 2025 Paper 13	Q5 (a),(b)	9
5	Oct/Nov 2023 Paper 12	Q9 (a),(b)	8
6	Oct/Nov 2023 Paper 13	Q9 (a),(b),(c),(d)	13
7	Oct/Nov 2025 Paper 12	Q3 (a),(b),(c)	10
8	Oct/Nov 2025 Paper 13	Q6 (a),(b),(c)	11

Total: 75 marks